

Recommendation Systems for Personalised Content Discovery

Disha | 24117045 | Mechanical engineering

Executive Summary

This report documents the development and evaluation of a recommendation system built on the Netflix Prize Dataset. The project was completed as a team effort: Matrix Factorization (SVD) serves as the primary recommendation model, while User-Based KNN and Item-Based KNN are implemented as comparative baselines.

The system was trained on a 10% user sample from all four Netflix data files (~9.4M ratings, 96,650 users, 17,765 movies) using a time-based 80/20 train-test split. All models were evaluated on RMSE and MAP@10 using relevance threshold ≥ 3.5 stars.

Model	Role	RMSE	MAP@10
SVD (Matrix Factorization)	Primary Model	0.9305	0.0235
User-Based KNN	Baseline	0.9957	0.0217
Item-Based KNN	Baseline	1.0240	0.0111

SVD outperformed both KNN approaches on all metrics — achieving the lowest RMSE (better rating prediction accuracy) and the highest MAP@10 (better ranking quality). User-KNN ranked second, and Item-KNN showed the weakest performance.

Dataset Overview

Source & Scale

The Netflix Prize Dataset was released as part of Netflix's 2006 open competition to improve its Cinematch recommendation algorithm. It remains one of the most widely used benchmarks in recommendation system research.

Attribute	Full Dataset	Sampled Dataset (Used)
Total Ratings	~100.4 Million	~9.4 Million
Users	480,189	96,650 (after ≥20 filter: 85,337)
Movies	17,770	17,765
Rating Scale	1–5 stars	1–5 stars
Date Range	1999–2005	1999-12-09 → 2005-12-31
Sparsity	~99.9%	98.86% (EDA file 1); 99.5% (model matrix)

Data Format

Each of the four data files (combined_data_1.txt through combined_data_4.txt) follows a specific format. Movie IDs appear on their own lines followed by a colon, and all subsequent rows until the next movie ID contain: User ID, Rating (1–5), Date (YYYY-MM-DD).

Sampling Strategy: 1 in every 10 users was retained to manage memory during model training (sample_every_n=10). For EDA, only combined_data_1.txt was analysed at sample_every_n=5.

Active User Filter: Users with fewer than 20 ratings were excluded, reducing the user count from 96,650 → 85,337.

Exploratory Data Analysis

EDA was conducted on combined_data_1.txt (24,053,764 ratings, 470,758 users, 4,499 movies from the first file). The following findings inform model design decisions and highlight important dataset characteristics.

Rating Distribution

The average rating across the dataset is 3.6 out of 5. The distribution is left-skewed, with ratings of 3 and 4 being the most common. This positive bias is a known characteristic of voluntary rating systems — users are more likely to rate content they enjoyed.

Rating counts: 1★: 1,118,186 | 2★: 2,439,073 | 3★: 6,904,181 | 4★: 8,085,741 | 5★: 5,506,583

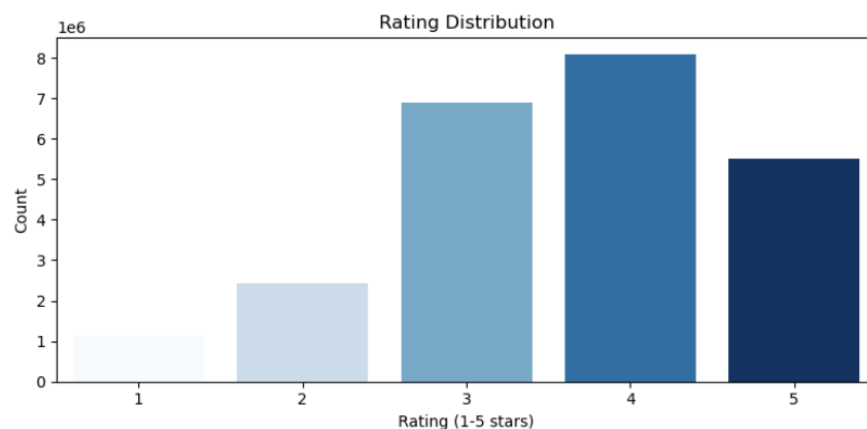


Figure 1: Rating distribution across all ratings in combined_data_1.txt

User & Movie Rating Averages

Per-user and per-movie average ratings were analysed to understand rating tendencies:

- 88.9% of users have an average rating above 3, confirming a dataset-wide positive bias.
- 23.7% of users average above 4 — these are highly generous raters.
- Only 5.7% of movies have an average rating above 4, making these genuinely top-rated titles.
- 10,862 users exclusively give 5-star ratings, indicating a non-negligible segment of uninformative raters.

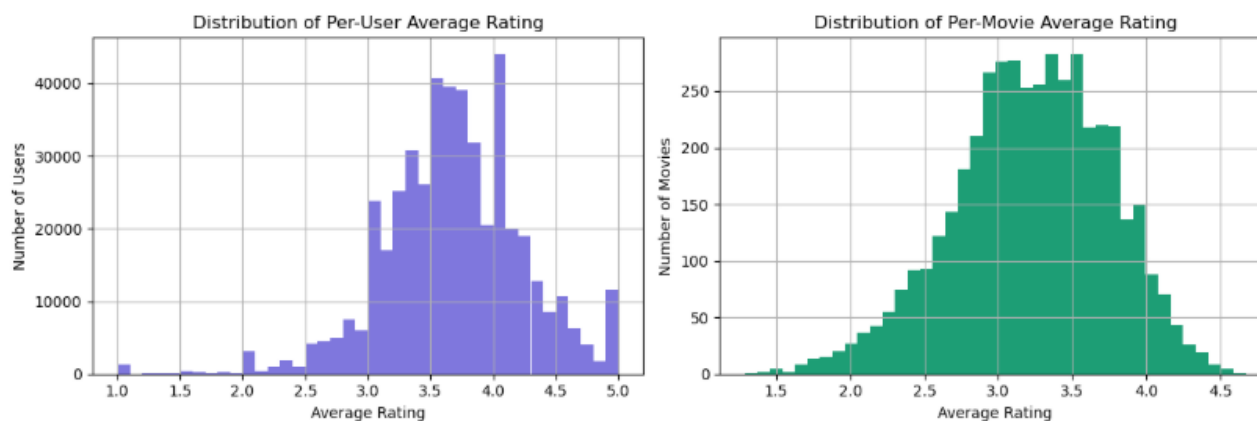


Figure 2: Distribution of per-user (left) and per-movie (right) average ratings

Content Popularity — Long Tail Effect

Movie popularity follows a strong long-tail distribution. A small fraction of titles account for the vast majority of ratings:

- Top 530 movies (11.8% of all movies) account for 80% of all ratings.
- The most-rated movie received 193,941 ratings; many movies have only a handful.
- This Pareto-like distribution directly motivates the candidate generation strategy used in model evaluation: recommendation generation was restricted to the top 200 most popular movies to keep inference feasible.

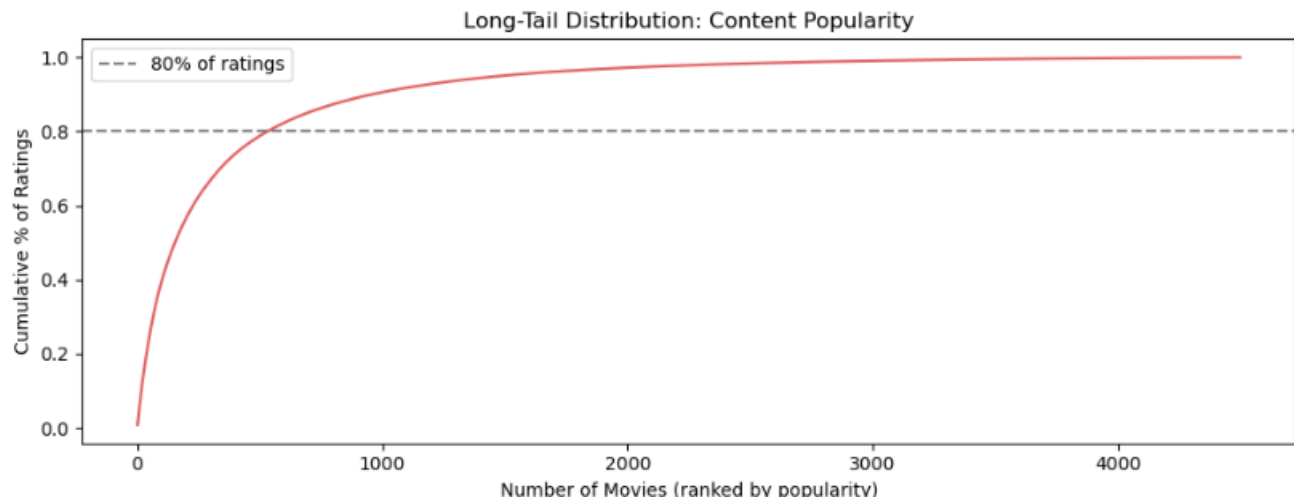


Figure 3: Cumulative % of ratings covered by top-N movies (long tail effect)

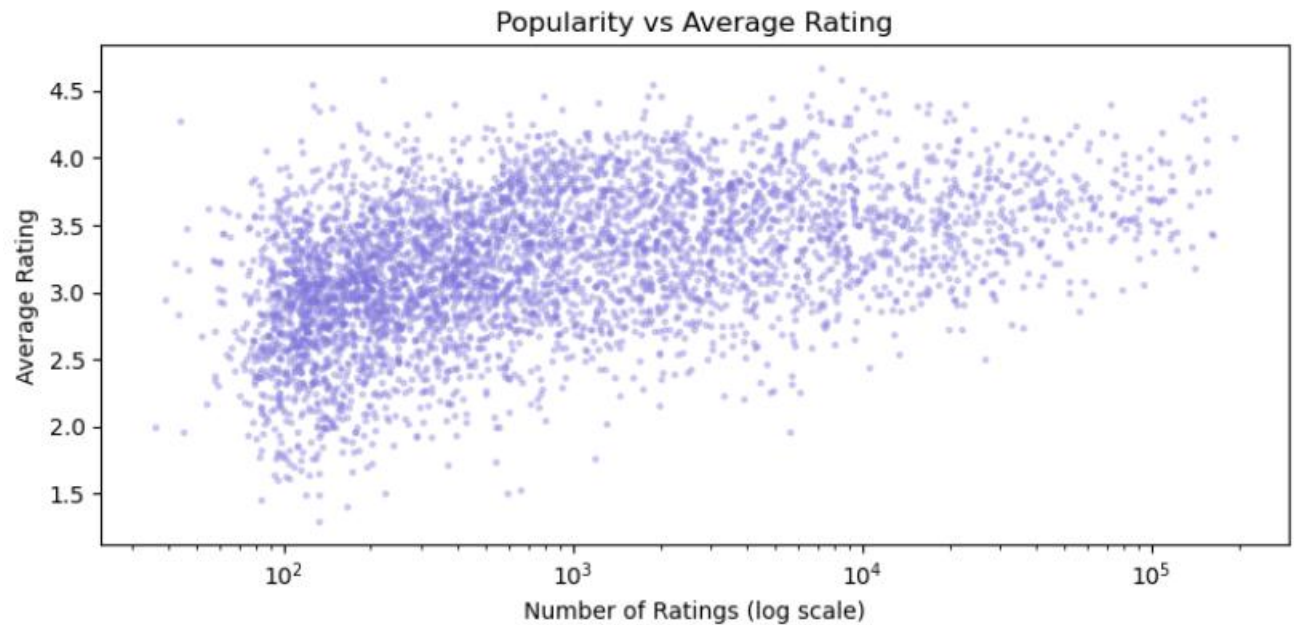


Figure 4: Movie popularity vs. average rating — log scale scatter plot

User Activity Patterns & Segmentation

Users were segmented into five activity tiers based on number of ratings:

Segment	Users	% of Ratings	Avg Rating	Std Dev
Casual (1–10)	140,127	3.0%	3.705	0.839
Light (11–50)	184,967	19.9%	3.702	0.999
Moderate (51–200)	124,836	51.1%	3.649	0.969

Segment	Users	% of Ratings	Avg Rating	Std Dev
Active (201–500)	19,734	22.9%	3.493	0.963
Power (500+)	<5,000	~3%	3.233	0.983

A notable pattern emerges: power users (highest activity) rate more critically (avg 3.23), while casual users are more generous (avg 3.70). This suggests that engagement depth correlates with rating discernment.

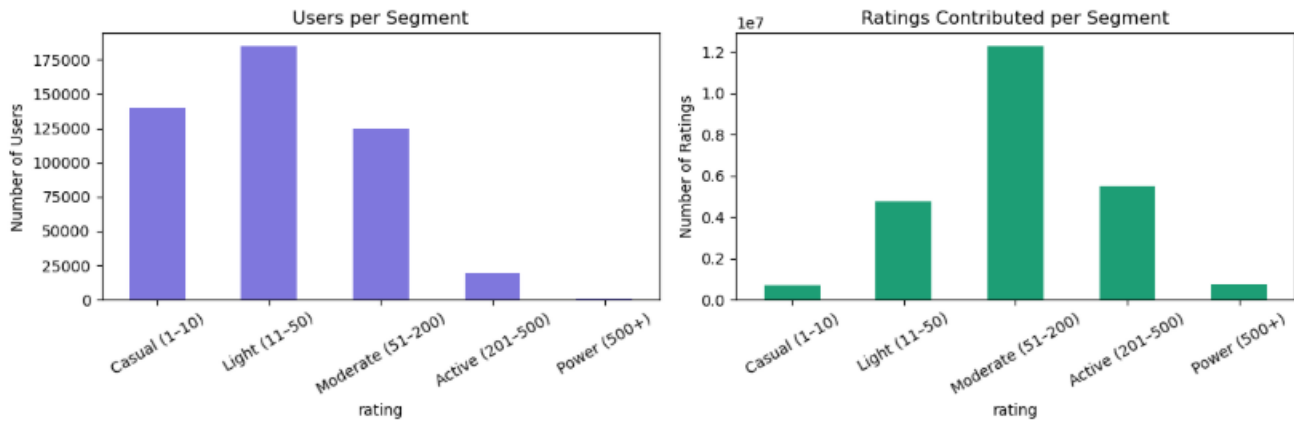


Figure 5: User count per segment (left) and % of total ratings per segment (right)

Data Sparsity

The user-item interaction matrix is extremely sparse — 98.86% of possible user-movie pairs have no rating. This is a fundamental challenge for collaborative filtering approaches, as the overlap between any two users or items is limited.

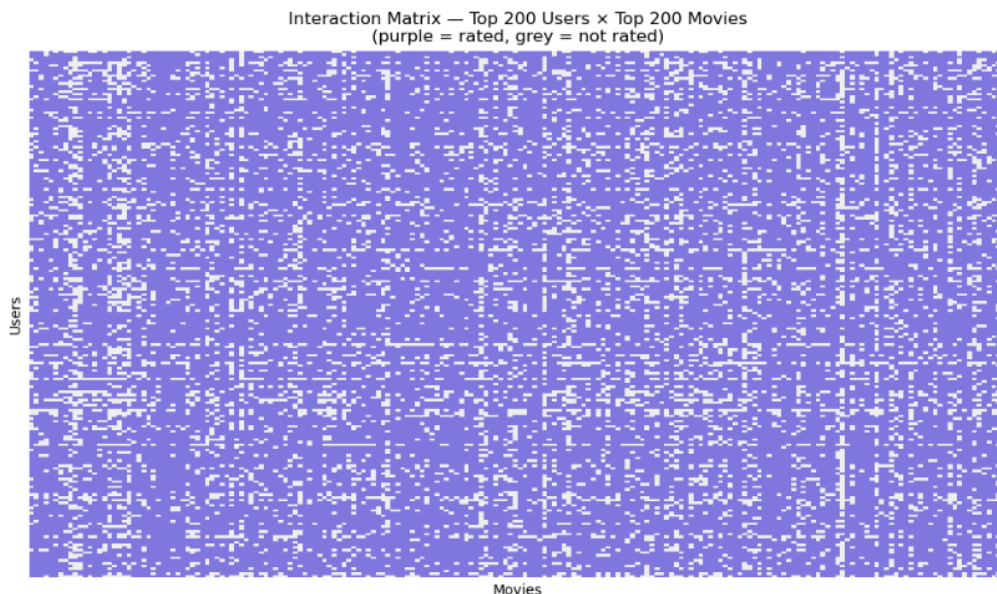


Figure 6: Interaction matrix heatmap — purple cells = rated, grey = missing

Cold Start Problem

Cold start is a well-known failure mode for collaborative filtering. Analysis shows:

- 13.9% of users have fewer than 5 ratings — these are near-impossible to personalise for.
- 27.6% of users have fewer than 10 ratings — limited signal available.
- No movies in this subset have fewer than 5 ratings (due to filtering), so the item cold-start problem is largely absent here.
- Mitigation: the ≥ 20 rating filter applied to training data removes the most problematic cold-start users from model evaluation.

Temporal Trends

Rating volume grew dramatically over time: from 426 ratings in 1999 to over 12.5 million in 2005. Average ratings also trended upward over the years, from 3.11 in 1999 to 3.67 in 2005 — possibly reflecting improving platform quality or a shift in the user base.

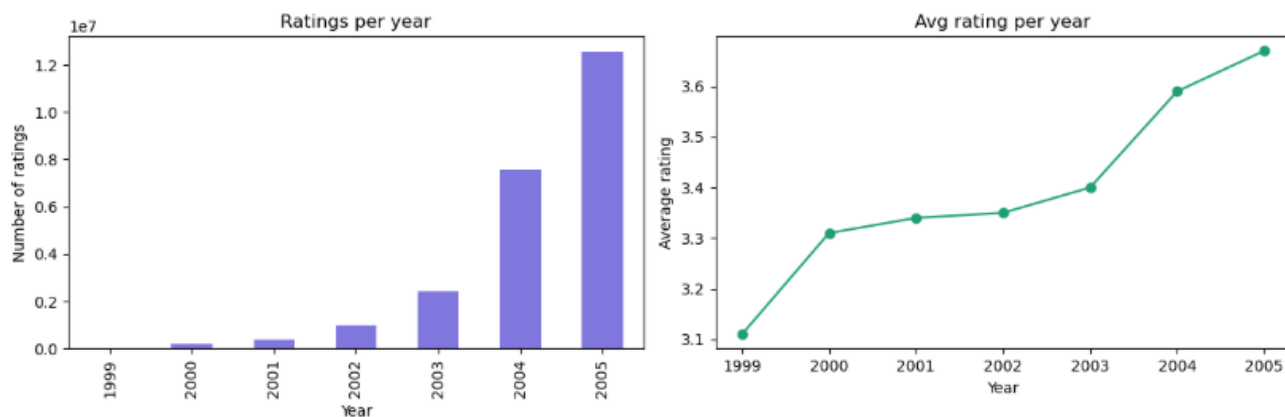


Figure 7: Annual rating volume (left) and average rating trend over time (right)

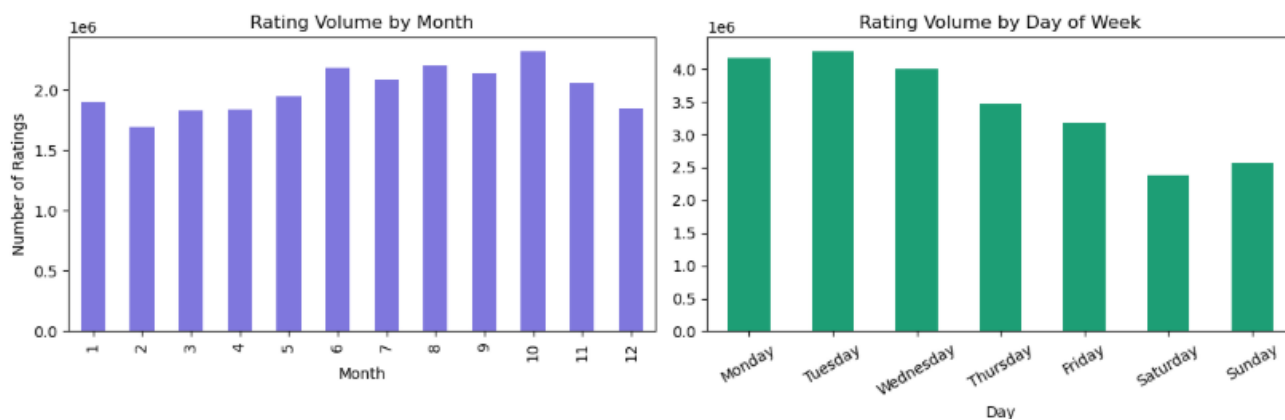


Figure 8: Rating volume by month (left) and by day of week (right)

Movie Decade Analysis

Content from the 1990s and 2000s dominates rating volume, which aligns with the platform's contemporaneous catalogue. Older films (pre-1970s) receive far fewer ratings but often maintain higher average scores — a likely survivor-bias effect where only iconic titles remain discoverable.

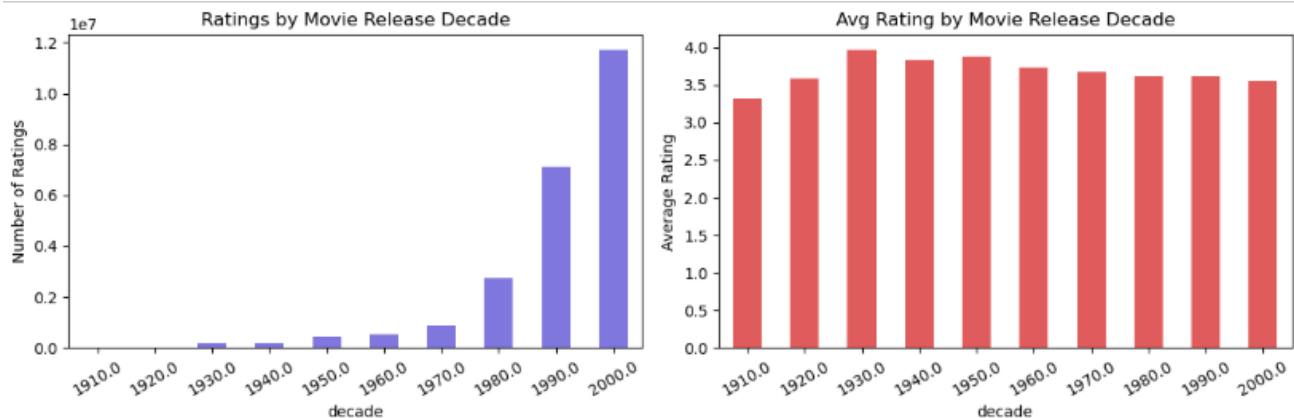


Figure 9: Rating volume (left) and average rating (right) by movie release decade

Methodology

Data Pipeline & Preprocessing

All four Netflix data files were parsed using a custom fast parser that reads movie IDs in-place and bulk-processes user ratings rows with pandas. To manage memory, 1 in every 10 users was sampled (`sample_every_n=10`). Users with fewer than 20 ratings were filtered out to focus on users with sufficient signal. After filtering, the final dataset contained approximately 9.4 million ratings.

Train-Test Split

A time-based 80/20 split was used. All ratings were sorted chronologically, and the 80th percentile date (2005-08-03) served as the cutoff. Ratings on or before this date formed the training set; ratings after formed the test set.

Rationale: The dataset spans six years (1999–2005). A temporal split prevents data leakage — the model never sees future ratings during training — and simulates realistic deployment conditions where recommendations are made for events not yet observed.

Training set: 7,537,987 ratings (85,337 users × 17,168 movies)

Test set: 1,086,209 ratings (filtered to users and movies present in training)

User-Item Matrix Construction

A dense user-item matrix of shape $(85,337 \times 17,168)$ was constructed from training data. Matrix sparsity was 99.5%. For similarity computations, scipy CSR sparse representations were used to avoid memory overflow during cosine similarity calculations.

Candidate Generation

All three models use a pre-computed candidate pool of the top 200 most-rated movies from the training set. For each user, recommendations are generated only over this pool (excluding already-

rated movies). This is a deliberate engineering trade-off: evaluating all 17,168 movies per user would be computationally infeasible at scale, and popular movies represent the most likely true positives in MAP@10 evaluation.

Recommendation Models

Matrix Factorization — SVD (Primary Model)

SVD (Singular Value Decomposition / Latent Factor Model) is the primary recommendation model. Rather than relying on direct user-user or item-item similarity, SVD learns dense latent factor representations for each user and movie.

Architecture

The model decomposes the rating matrix $R \approx \mu + bu + bi + P \cdot Q^T$, where:

- μ = global mean rating
- bu = user bias (per-user deviation from global mean)
- bi = item bias (per-movie deviation from global mean)
- P = user latent factor matrix ($85,337 \times 50$)
- Q = item latent factor matrix ($17,168 \times 50$)

Training

The model was trained with Stochastic Gradient Descent (SGD) over 20 epochs. On each epoch, all training ratings are shuffled and processed row-by-row. For each rating, the error (true – predicted) is used to update all parameters simultaneously.

Hyperparameter	Value	Description
n_factors	50	Dimensionality of latent space
n_epochs	20	Full passes over training data
lr (learning rate)	0.005	SGD step size
reg (regularisation)	0.02	L2 penalty to prevent overfitting

Strengths & Limitations

- Strengths: Captures global latent structure (genres, styles) not visible in local neighbourhoods. Handles sparsity well. Learns user and item biases explicitly, improving prediction accuracy on skewed data.
- Limitations: Computationally expensive — 20 epochs $\times$ 7.5M ratings = ~150M parameter updates. No interpretability: latent factors have no semantic meaning. Performance constrained by candidate pool size at evaluation.

User-Based Collaborative Filtering (KNN)

User-Based KNN finds the k most similar users to the target user and predicts ratings as a similarity-weighted average of their known ratings. Similarity is computed using cosine similarity on the sparse user-item matrix.

Implementation Details

- $k = 20$ nearest neighbours
- Similarity: cosine similarity computed on-demand (not pre-computed matrix) to avoid memory errors on an 85,337-user matrix
- Rating prediction: weighted average of neighbour ratings for the target movie, using only neighbours who have rated that movie
- Fallback: global mean is used when no neighbours have rated a movie

Strengths & Limitations

- Strengths: Intuitive — finds users with shared taste. Works well when users have sufficient rating overlap.
- Limitations: On-demand similarity computation is slow (cosine against all 85,337 users per prediction). Extremely sparse data limits effective neighbours. Popularity bias: tends to recommend widely-rated movies.

Item-Based Collaborative Filtering (KNN)

Item-Based KNN predicts ratings by finding movies similar to those the user has already rated. Similarity is computed between item vectors (columns of the user-item matrix).

Implementation Details

- $k = 20$ nearest neighbours
- For evaluation, item similarity was pre-computed only within the top 200 candidate pool (200×200 matrix) — this dramatically accelerated recommendation generation from hours to minutes
- Rating prediction: weighted average of user's ratings for similar items

Strengths & Limitations

- Strengths: Item similarity is more stable than user similarity (items don't change; user preferences drift). Faster inference at test time with the pre-computed similarity matrix.
 - Limitations: Worst RMSE and MAP@10 among all three models. May be biased toward mainstream content similarities. The 200-movie restriction further limits diversity of recommendations.
-

Evaluation

Evaluation Protocol

- Test sample for RMSE: 1,000 randomly sampled ratings from the test set
- Test users for MAP@10: First 500 unique users in the test set (User-KNN and SVD); 200 users for Item-KNN (computational constraint)
- Relevance threshold: A movie is considered relevant if actual rating ≥ 3.5
- Candidate pool: Top 200 most popular training movies, excluding already-rated movies per user

Metric Definitions

RMSE (Root Mean Squared Error)

Measures point-prediction accuracy. For each (user, movie) pair in the test sample, the model predicts a rating which is compared to the true rating. Lower RMSE = better rating prediction.

MAP@10 (Mean Average Precision at 10)

Measures recommendation ranking quality. For each test user, the top 10 recommended movies are compared against the user's actual relevant movies in the test set (rating ≥ 3.5). Average Precision rewards recommendations that appear early in the ranked list. MAP@10 averages this across all users. Higher MAP@10 = better ranking.

Results

Model	RMSE ↓	MAP@10 ↑	Test Users (MAP)
SVD (Primary)	0.9305	0.0235	500
User-Based KNN	0.9957	0.0217	500
Item-Based KNN	1.0240	0.0111	200

Discussion

SVD achieves the best results on both metrics. The gap in RMSE between SVD (0.9305) and User-KNN (0.9957) is approximately 0.065 — a meaningful improvement that reflects SVD's ability to learn global latent structure rather than relying on local neighbourhood averages.

The MAP@10 values are low across all models (range: 0.011–0.023). This is expected given:

- The candidate pool is restricted to the top 200 popular movies, not the full catalogue
- Only 500 test users are evaluated; many may have very few test-set relevant movies
- The dataset is extremely sparse, making it hard to find strong preference signals for less-active users

Item-KNN performs worst on MAP@10 (0.0111), approximately half that of User-KNN (0.0217). A likely reason is that item-item similarity within the top 200 popular movies is less differentiated — popular movies tend to be broadly appealing and share similar user bases, reducing the specificity of item-based signals.

Recommendation Analysis

Success Case — User 698451

This user was successfully profiled based on their training history. They had rated action/adventure classics highly:

Liked in Training: Star Wars: Episode VI — Return of the Jedi, Gladiator, The Lord of the Rings: The Fellowship of the Ring (Extended Edition), GoodFellas: Special Edition, Happy Gilmore

Recommended: Lord of the Rings: The Fellowship of the Ring, Lord of the Rings: The Two Towers, The Godfather, Raiders of the Lost Ark, Saving Private Ryan...

Hits: 4 out of 10 recommendations matched test-set relevant movies

This is a textbook success case: the user has a clear preference for epic action-adventure and critically acclaimed dramas, and the model correctly identifies related content. The recommendation list is coherent and interpretable — even without reading a latent factor, a human reviewer would recognise the logic.

Failure Case

Failure cases occur primarily for users with fewer than 25 training ratings. These users have provided insufficient signal for the model to build a reliable preference profile. In the test set, their relevant movies (rating ≥ 3.5) receive no hits from the top-10 recommendations.

Root causes of failure:

- Sparse training history: with fewer interactions, user similarity vectors are unreliable
- Popular-movie bias: all models are constrained to recommend from the top 200 movies, which may not align with niche preferences of low-activity users
- Cold start boundary: users with <20 ratings are excluded entirely; the 20–25 rating range is the next most vulnerable group
- Temporal dynamics: a user's preferences in 2005 (test set) may not align with what they rated in earlier years (training set)

Key Observations

- All three models share the same candidate pool (top 200 movies), which means MAP@10 comparisons measure the models' ability to rank within that pool — not their ability to surface diverse or long-tail recommendations.
 - SVD's advantage likely comes from its bias terms: it explicitly models the tendency of certain movies to be universally over/under-rated, and certain users to rate consistently higher or lower. KNN models cannot separate these effects.
 - User-KNN outperforms Item-KNN on both metrics. This is consistent with the research literature for datasets where users have diverse but overlapping preferences — user-level signal tends to be richer.
 - The MAP@10 scores suggest all models struggle to rank the truly relevant items in the top 10 most of the time. This is a candidate pool problem as much as a model problem — expanding the pool or using a two-stage retrieve-then-rank approach would likely improve MAP.
-

Conclusion

Summary

This project implemented and evaluated three recommendation models on the Netflix Prize Dataset. The primary model, Matrix Factorization via SGD-trained SVD, achieved the best performance across both RMSE and MAP@10. User-Based KNN served as a strong secondary baseline, while Item-Based KNN showed weaker results on this dataset configuration.

— End of Report —